

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

**MODUL XIII
PENGENALAN MULTI LINKED LIST**



Disusun Oleh :

NAMA : Rikza Nur Zaki

NIM : 103112430030

Dosen

FAHRUDIN MUKTI WIBOWO

**PROGRAM STUDI STRUKTUR DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025**

A. Teori Dasar Multi Linked List

Multi linked list, atau multilist, adalah struktur data di mana node tidak hanya menunjuk ke node berikutnya seperti single linked list, melainkan memiliki dua atau lebih pointer untuk menghubungkan node parent ke child atau sebaliknya. Ini memungkinkan representasi relasi data seperti tree atau graph, di mana satu node parent bisa memiliki banyak child node. Setiap node biasanya berisi data, pointer ke parent, dan pointer ke child (atau multiple child). Mendukung relasi 1-N (satu parent banyak child) atau M-N (banyak ke banyak) melalui variasi bentuk seperti list relasi tambahan. Lebih fleksibel daripada doubly linked list, tapi memerlukan manajemen memori ekstra untuk menghindari memory leak saat delete, kelebihan termasuk kemudahan menghubungkan relasi data dinamis, sementara kekurangannya adalah kompleksitas pencarian child dan overhead memori.

B. Guided (berisi screenshot source code & output program disertai penjelasannya)

Guided 1

```

#include <iostream>
#include <string>
using namespace std;

struct ChildNode {
    string info;
    ChildNode *next;
    ChildNode *prev;
};

struct ParentNode {
    string info;
    ChildNode *childHead;
    ParentNode *next;
    ParentNode *prev;
};

ParentNode *createParent(string info) {
    ParentNode *newNode = new ParentNode;
    newNode->info = info;
    newNode->childHead = NULL;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}

ChildNode *createChild(string info) {
    ChildNode *newNode = new ChildNode;
    newNode->info = info;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}

void insertParent(ParentNode *&head, string info) {
    ParentNode *newNode = createParent(info);
    if (head == NULL) {
        head = newNode;
    } else {
        ParentNode *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
}

```

```

void insertChild(ParentNode *head, string parentInfo, string childInfo) {
    ParentNode *p = head;
    while (p != NULL && p->info != parentInfo) {
        p = p->next;
    }

    if (p != NULL) {
        ChildNode *newChild = createChild(childInfo);
        if (p->childHead == NULL) {
            p->childHead = newChild;
        } else {
            ChildNode *c = p->childHead;
            while (c->next != NULL) {
                c = c->next;
            }
            c->next = newChild;
            newChild->prev = c;
        }
    }
}

void printAll(ParentNode *head) {
    while (head != NULL) {
        cout << head->info;
        ChildNode *c = head->childHead;
        while (c != NULL) {
            cout << " -> " << c->info;
            c = c->next;
        }
        cout << endl;
        head = head->next;
    }
}

void updateParent(ParentNode *head, string oldInfo, string newInfo) {
    ParentNode *p = head;
    while (p != NULL) {
        if (p->info == oldInfo) {
            p->info = newInfo;
            return;
        }
        p = p->next;
    }
}

```

```

void updateChild(ParentNode *head, string parentInfo, string oldChildInfo, string newChildInfo) {
    ParentNode *p = head;
    while (p != NULL && p->info != parentInfo) {
        p = p->next;
    }
    if (p != NULL) {
        ChildNode *c = p->childHead;
        while (c != NULL) {
            if (c->info == oldChildInfo) {
                c->info = newChildInfo;
                return;
            }
            c = c->next;
        }
    }
}

void deleteChild(ParentNode *head, string parentInfo, string childInfo) {
    ParentNode *p = head;
    while (p != NULL && p->info != parentInfo) {
        p = p->next;
    }
    if (p != NULL) {
        ChildNode *c = p->childHead;
        while (c != NULL) {
            if (c->info == childInfo) {
                if (c == p->childHead) {
                    p->childHead = c->next;
                    if (p->childHead != NULL) {
                        p->childHead->prev = NULL;
                    }
                } else {
                    c->prev->next = c->next;
                    if (c->next != NULL) {
                        c->next->prev = c->prev;
                    }
                }
                delete c;
                return;
            }
            c = c->next;
        }
    }
}

```

```

void deleteParent(ParentNode *&head, string info) {
    ParentNode *p = head;
    while (p != NULL) {
        if (p->info == info) {
            ChildNode *c = p->childHead;
            while (c != NULL) {
                ChildNode *tempC = c;
                c = c->next;
                delete tempC;
            }

            if (p == head) {
                head = p->next;
                if (head != NULL) {
                    head->prev = NULL;
                }
            } else {
                p->prev->next = p->next;
                if (p->next != NULL) {
                    p->next->prev = p->prev;
                }
            }
            delete p;
            return;
        }
        p = p->next;
    }
}

int main() {
    ParentNode *list = NULL;

    insertParent(list, "Parent A");
    insertParent(list, "Parent B");
    insertParent(list, "Parent C");

    cout << "\nSetelah InsertParent:" << endl;
    printAll(list);

    insertChild(list, "Parent A", "Child A1");
    insertChild(list, "Parent A", "Child A2");
    insertChild(list, "Parent B", "Child B1");

    cout << "\nSetelah InsertChild:" << endl;
    printAll(list);

    updateParent(list, "Parent B", "Parent B*");
    updateChild(list, "Parent A", "Child A1", "Child A1*");

    cout << "\nSetelah Update:" << endl;
    printAll(list);

    deleteChild(list, "Parent A", "Child A2");
    deleteParent(list, "Parent C");

    cout << "\nSetelah Delete:" << endl;
    printAll(list);
    return 0;
}

```

Screenshots Output

```
Setelah InsertParent:
Parent A
Parent B
Parent C

Setelah InsertChild:
Parent A -> Child A1 -> Child A2
Parent B -> Child B1
Parent C

Setelah Update:
Parent A -> Child A1* -> Child A2
Parent B* -> Child B1
Parent C

Setelah Delete:
Parent A -> Child A1*
Parent B* -> Child B1
```

Deskripsi:

Program diatas mengimplementasikan multilevel linked list berbasis double linked list, di mana setiap ParentNode menyimpan data, pointer ke parent lain (next, prev), serta pointer childHead yang mengarah ke linked list ChildNode. Struktur ChildNode juga menggunakan next dan prev untuk memungkinkan penelusuran dua arah. Program menyediakan fungsi untuk membuat node parent dan child (createParent, createChild), menambahkan parent dan child (insertParent, insertChild), menampilkan seluruh data parent beserta child nya (printAll), memperbarui data parent dan child (updateParent, updateChild), serta menghapus child tertentu (deleteChild) dan parent dengan seluruh child nya (deleteParent). Pada fungsi main, seluruh fitur tersebut diuji melalui proses insert, update, delete, dan penampilan data, sehingga merepresentasikan hasil seperti di gambar ss atas.

- C. Unguided/Tugas (berisi screenshot source code & output program disertai penjelasannya)

Unguided 1

circualist.h

```

#ifndef CIRCULARLIST_H_INCLUDED
#define CIRCULARLIST_H_INCLUDED

#include <iostream>
#include <string>

using namespace std;

struct mahasiswa {
    string nama;
    string nim;
    char jenis_kelamin;
    float ipk;
};

typedef mahasiswa infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

void createList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void insertFirst(List &L, address P);
void insertAfter(List &L, address Prec, address P);
void insertLast(List &L, address P);
void deleteFirst(List &L, address &P);
void deleteAfter(List &L, address Prec, address &P);
void deleteLast(List &L, address &P);
address findElm(List L, infotype x);
void printInfo(List L);

address createData(string nama, string nim, char jenis_kelamin, float ipk);

#endif

```

circualist.cpp

```

#include "circularlist.h"

void createList(List &L) {
    L.First = NULL;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = NULL;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = NULL;
}

void insertFirst(List &L, address P) {
    if (L.First == NULL) {
        L.First = P;
        P->next = L.First;
    } else {
        address Q = L.First;
        while (Q->next != L.First) {
            Q = Q->next;
        }
        P->next = L.First;
        Q->next = P;
        L.First = P;
    }
}

void insertLast(List &L, address P) {
    if (L.First == NULL) {
        insertFirst(L, P);
    } else {
        address Q = L.First;
        while (Q->next != L.First) {
            Q = Q->next;
        }
        Q->next = P;
        P->next = L.First;
    }
}

```

```

void insertAfter(List &L, address Prec, address P) {
    if (Prec != NULL) {
        P->next = Prec->next;
        Prec->next = P;
    }
}

void deleteFirst(List &L, address &P) {
    if (L.First != NULL) {
        P = L.First;
        if (P->next == L.First) {
            L.First = NULL;
        } else {
            address Q = L.First;
            while (Q->next != L.First) {
                Q = Q->next;
            }
            L.First = P->next;
            Q->next = L.First;
        }
        P->next = NULL;
    }
}

void deleteLast(List &L, address &P) {
    if (L.First != NULL) {
        if (L.First->next == L.First) {
            deleteFirst(L, P);
        } else {
            address Q = L.First;
            while (Q->next->next != L.First) {
                Q = Q->next;
            }
            P = Q->next;
            Q->next = L.First;
            P->next = NULL;
        }
    }
}

```

```

void deleteAfter(List &L, address Prec, address &P) {
    if (Prec != NULL && Prec->next != L.First) {
        P = Prec->next;
        Prec->next = P->next;
        P->next = NULL;
    }
}

address findElm(List L, infotype x) {
    if (L.First == NULL) return NULL;
    address P = L.First;
    do {
        if (P->info.nim == x.nim) {
            return P;
        }
        P = P->next;
    } while (P != L.First);
    return NULL;
}

void printInfo(List L) {
    if (L.First == NULL) {
        cout << "List Kosong" << endl;
        return;
    }
    address P = L.First;
    do {
        cout << "Nama : " << P->info.nama << endl;
        cout << "NIM : " << P->info.nim << endl;
        cout << "L/P : " << P->info.jenis_kelamin << endl;
        cout << "IPK : " << P->info.ipk << endl;
        cout << "-----" << endl;
        P = P->next;
    } while (P != L.First);
}

address createData(string nama, string nim, char jenis_kelamin, float ipk) {
    infotype x;
    x.nama = nama;
    x.nim = nim;
    x.jenis_kelamin = jenis_kelamin;
    x.ipk = ipk;
    return alokasi(x);
}

```

main.cpp

```

#include "circularlist.h"
#include "circularlist.cpp"

int main() {
    List L;
    address P1, P2;
    infotype x;

    createList(L);

    cout << "coba insert first, last, dan after" << endl;

    P1 = createData("Danu", "04", 'L', 4.0);
    insertFirst(L, P1);

    P1 = createData("Fahmi", "06", 'L', 3.45);
    insertLast(L, P1);

    P1 = createData("Bobi", "02", 'L', 3.71);
    insertFirst(L, P1);

    P1 = createData("Ali", "01", 'L', 3.3);
    insertFirst(L, P1);

    P1 = createData("Gita", "07", 'P', 3.75);
    insertLast(L, P1);

    x.nim = "04";
    P1 = findElm(L, x);
    P2 = createData("Cindi", "03", 'P', 3.5);
    insertAfter(L, P1, P2);

    x.nim = "02";
    P1 = findElm(L, x);
    P2 = createData("Hilmi", "08", 'P', 3.3);
    insertAfter(L, P1, P2);

    x.nim = "01";
    P1 = findElm(L, x);
    P2 = createData("Eli", "05", 'P', 3.4);
    insertAfter(L, P1, P2);

    printInfo(L);

    return 0;
}

```

Screenshots Output

```

coba insert first, last, dan after
Nama : Ali
NIM : 01
L/P : L
IPK : 3.3
-----
Nama : Eli
NIM : 05
L/P : P
IPK : 3.4
-----
Nama : Bobi
NIM : 02
L/P : L
IPK : 3.71
-----
Nama : Hilmi
NIM : 08
L/P : P
IPK : 3.3
-----
Nama : Danu
NIM : 04
L/P : L
IPK : 4
-----
Nama : Cindi
NIM : 03
L/P : P
IPK : 3.5
-----
Nama : Fahmi
NIM : 06
L/P : L
IPK : 3.45
-----
Nama : Gita
NIM : 07
L/P : P
IPK : 3.75
-----

```

Deskripsi:

Program diatas mengimplementasikan Circular Singly Linked List untuk menyimpan data mahasiswa, di mana setiap node (ElmList) berisi informasi mahasiswa (nama, NIM, jenis kelamin, dan IPK) serta pointer next yang kembali ke node pertama. Struktur List menyimpan pointer First sebagai penanda awal list. Fungsi createList digunakan untuk menginisialisasi list agar kosong, sedangkan alokasi dan dealokasi berfungsi untuk mengelola memori node. Penambahan data dilakukan melalui insertFirst, insertLast, dan insertAfter, yang masing-masing terdapat node di awal, akhir, atau setelah node tertentu. Proses penghapusan node disediakan melalui deleteFirst, deleteLast, dan deleteAfter, yang menangani kondisi khusus seperti list berisi satu elemen maupun banyak elemen. Fungsi findElm digunakan untuk mencari node mahasiswa berdasarkan NIM dengan menelusuri list secara melingkar, sementara printInfo menampilkan seluruh data mahasiswa dalam list hingga kembali ke node pertama. Fungsi createData mempermudah pembuatan data mahasiswa sekaligus alokasi node baru. Pada main, seluruh fungsi tersebut diujikan dan menghasilkan hasil ss diatas.

D. Kesimpulan

Berdasarkan laporan praktikum tersebut, dapat disimpulkan bahwa Multi Linked List merupakan struktur data dinamis yang lebih fleksibel dibandingkan doubly linked list karena mampu merepresentasikan relasi data yang kompleks, seperti hubungan satu-ke-banyak (1-N) atau banyak-ke-banyak (M-N) melalui penggunaan dua atau lebih pointer untuk menghubungkan node parent dan child. Implementasi praktikum menunjukkan bahwa struktur ini memungkinkan operasi penambahan, pembaruan, dan penghapusan data secara terorganisir, baik pada level induk maupun anak, meskipun memerlukan manajemen memori yang lebih teliti untuk mencegah terjadinya kebocoran memori (memory leak). Selain itu, laporan ini juga mendemonstrasikan penerapan Circular Singly Linked List untuk mengelola data mahasiswa.

E. Referensi

- [Knuth, Donald](#) (1998). *The Art of Computer Programming*. Vol. 3: *Sorting and Searching* (2nd ed.). Addison-Wesley. p. 547. [ISBN 978-0-201-89685-5](#).
- Chris Okasaki (1995). "Purely Functional Random-Access Lists". *Proceedings of the Seventh International Conference on Functional Programming Languages and Computer Architecture*: 86–95. [doi:10.1145/224164.224187](#)
- Ford, William; Topp, William (2002). *Data Structures with C++ using STL* (Second ed.). Prentice-Hall. pp. 466–467. [ISBN 0-13-085850-1](#).